

Comparative Analysis of Dijkstra and A* Algorithms for Determining the Shortest Route

Ardiansyah^{1)*}, Abdul Muin Nasution²⁾, Muhammad Iqbal³⁾

¹⁾³⁾ Information Technology, Faculty of Computational Science and Digital Intelligence, Universitas Pembangunan Panca Budi, Medan, Indonesia

²⁾ Information Technology, Faculty of Computer Science and Information Technology, Randwick International Research and Analysis Institute, Medan, Indonesia

¹⁾ardiansyah.binjai@gmail.com

²⁾muinnasution@gmail.com

³⁾muhammadiqbalpb@gmail.com

Article history:

Received 29 Nov 2025;
Revised 01 Dec 2025;
Accepted 08 Dec 2025;
Available online 10 Dec 2025

Keywords:

A* algorithm
Dijkstra algorithm
Graph theory
Heuristic search
Shortest path

Abstract

This study presents a comparative analysis of Dijkstra and A* algorithms for determining the shortest route in an urban road network scenario, specifically from SMKN 9 Medan to Gramedia Gajah Mada, Medan. The road network is modeled as a weighted graph, where nodes represent key locations, and edges represent inter-node distances derived from Google Maps. Three alternative routes are evaluated based on inter-node distances and direct heuristic distances to the destination. Dijkstra's algorithm, an uninformed search method, guarantees optimality by exhaustively exploring all possible paths with non-negative weights. In contrast, the A* algorithm incorporates a heuristic function that estimates the remaining distance to the goal, enhancing search efficiency by focusing on the most promising paths. Both algorithms are applied to the same graph data for a fair comparison, with performance metrics including total route distance, number of nodes explored, and computational efficiency. The results show that while both algorithms identify the same optimal route (A–B–E–G, 5.7 km), A* outperforms Dijkstra in terms of computational efficiency, exploring fewer nodes and requiring less computation time. These findings suggest that while Dijkstra remains reliable for smaller networks, A* is better suited for real-world navigation applications where efficiency and scalability are critical. This study provides empirical evidence supporting the use of heuristic-based algorithms in urban route planning systems.

I. INTRODUCTION

Shortest path determination is a fundamental problem in computer science and has long served as a cornerstone for research in graph theory, artificial intelligence, and transportation systems. In practical applications, shortest path algorithms are widely used in navigation systems, geographic information systems (GIS), logistics optimization, emergency response, robotics, and smart city infrastructures. Road networks are commonly represented as weighted graphs, where nodes correspond to locations or intersections and edges represent roads associated with weights such as distance, travel time, or cost. Within this representation, the efficiency and accuracy of pathfinding algorithms directly affect system performance and user experience, particularly in urban environments characterized by dense networks and frequent route queries [1], [2].

Among classical shortest path algorithms, Dijkstra's algorithm remains one of the most reliable and widely implemented methods for graphs with non-negative edge weights. The algorithm systematically explores all possible paths by expanding nodes with the smallest tentative distance, guaranteeing that the resulting path is globally optimal [3], [4], [5]. Due to its deterministic nature and mathematical correctness, Dijkstra's algorithm is often used as a benchmark in shortest path research. However, its exhaustive exploration strategy leads to increased computational cost as the size and complexity of the graph grow. This limitation becomes particularly

* Corresponding author

significant in large-scale urban road networks, where the number of nodes and edges can be substantial and real-time responsiveness is required [6].

To address these efficiency challenges, heuristic-based search algorithms have been introduced, with the A* algorithm being one of the most prominent examples. A* extends the principles of Dijkstra's algorithm by incorporating a heuristic function that estimates the remaining cost from a node to the destination. By combining the actual cost from the start node with this heuristic estimate, A* prioritizes nodes that are more likely to lead toward the goal, thereby reducing unnecessary exploration. When the heuristic function is admissible and consistent, A* guarantees optimal solutions while significantly improving computational efficiency compared to uninformed search methods [7], [8].

Recent studies have demonstrated that A* consistently explores fewer nodes and achieves faster execution times than Dijkstra's algorithm when applied to real-world transportation networks. This advantage is particularly evident in urban navigation scenarios, where heuristic information such as Euclidean distance or estimated travel time can effectively guide the search process. Improved variants of A* have been proposed to further enhance performance by integrating road attributes, spatial constraints, and environment-specific factors, such as road accessibility and traffic conditions [9], [10]. These developments highlight the growing importance of heuristic design in determining the practical effectiveness of shortest path algorithms.

Despite the proven efficiency of A*, comparative evaluations with Dijkstra's algorithm remain relevant, especially when the underlying data are static and derived from commonly used mapping services. Many navigation and routing studies rely on distance or travel-time estimates obtained from platforms such as Google Maps. While such data sources provide accessibility and realism, they may introduce uncertainties related to estimation accuracy, temporal variation, and regional bias [11]. Empirical evidence indicates that discrepancies can arise between API-based estimates and actual travel conditions, particularly in developing urban contexts. Therefore, controlled comparisons using identical datasets are essential to ensure that observed performance differences stem from algorithmic characteristics rather than data inconsistencies [12].

In addition to efficiency considerations, modern route planning research increasingly emphasizes application-specific optimization goals. Shortest distance does not always correspond to the most desirable route when factors such as congestion, environmental impact, or safety are considered. Studies on eco-routing and sustainable transportation show that alternative routes may yield lower emissions or energy consumption despite longer distances. This trend underscores the need for flexible routing frameworks in which classical algorithms like Dijkstra and A* serve as foundational baselines before being extended with multi-criteria or learning-based approaches [13].

From an implementation perspective, maintaining algorithmic simplicity and interpretability remains important, particularly for educational applications and decision-support systems. Dijkstra's algorithm, with its straightforward relaxation process, offers clarity and robustness, making it suitable for smaller networks and instructional contexts. Conversely, A* provides a practical balance between optimality and efficiency, which is advantageous for real-world navigation systems that demand faster response times and reduced computational overhead. Hybrid approaches, such as graph clustering or hierarchical routing, further enhance performance while preserving the core principles of classical shortest path algorithms [2].

Given these considerations, a systematic comparison between Dijkstra and A* algorithms remains valuable for understanding their relative strengths and limitations under realistic conditions. Evaluating both algorithms using the same road network representation and distance data allows for a fair assessment of efficiency, optimality, and scalability. Such comparative studies also provide empirical baselines that can inform the integration of more advanced techniques, including reinforcement learning and adaptive routing methods designed for dynamic environments [14], [15]. Therefore, this study focuses on a comparative analysis of Dijkstra and A* algorithms for determining the shortest route within an urban road network scenario. By modeling the road network as a weighted graph and applying both algorithms to identical distance data, this research aims to assess route optimality, computational efficiency, and node exploration behavior. The results are expected to contribute practical insights for the development of intelligent navigation systems and to reinforce the role of heuristic-based algorithms as efficient solutions for urban route planning applications.

II. RELATED WORKS/LITERATURE REVIEW

Shortest path computation has remained a central research topic in computer science due to its foundational role in graph theory and its extensive application in transportation systems, geographic information systems, and intelligent navigation platforms. Road networks are commonly represented as weighted graphs, where nodes denote locations or intersections and edges represent roads with associated costs such as distance or travel time. This abstraction enables systematic evaluation of routing algorithms and provides a consistent framework for comparing optimality and computational efficiency across different methods [1].

Among classical algorithms, Dijkstra's algorithm continues to serve as a fundamental baseline for shortest path determination in graphs with non-negative edge weights. Its deterministic relaxation process guarantees optimality, making it a reliable reference point in both theoretical and applied studies. However, recent literature

consistently reports that Dijkstra's exhaustive exploration leads to higher computational overhead when applied to dense or large-scale urban networks, particularly when multiple routing queries are required [6]. As urban navigation systems increasingly demand fast response times, this limitation has motivated the adoption of more efficient heuristic-based approaches.

The A* algorithm addresses this challenge by integrating a heuristic function that estimates the remaining cost from a given node to the destination, thereby guiding the search toward promising regions of the graph. When the heuristic is admissible and consistent, A* preserves optimality while significantly reducing the number of explored nodes. Empirical evaluations on spatial road networks show that A* consistently outperforms Dijkstra in terms of execution time and search efficiency, especially when Euclidean or straight-line distance heuristics are applied [7]. This efficiency gain has positioned A* as a preferred choice for practical navigation systems.

Recent research has further extended the A* framework through algorithmic refinements tailored to specific environments. Improved A* variants incorporate road attributes, spatial constraints, or contextual information to enhance routing relevance in complex scenarios such as emergency response and off-road navigation. These studies demonstrate that domain-aware heuristics can substantially narrow the search space and improve convergence speed without sacrificing route optimality [9]. In multi-modal urban transportation systems, enhanced A* implementations have also been shown to handle heterogeneous network characteristics more effectively than uninformed search methods [1].

Beyond static shortest path computation, contemporary research increasingly focuses on dynamic routing, where edge weights vary over time due to traffic conditions, incidents, or signal control. In such settings, classical algorithms like Dijkstra and A* are often used as baselines against which adaptive or learning-based methods are compared. Reinforcement learning approaches, for example, estimate shortest paths under changing network conditions and demonstrate improved adaptability compared to static algorithms, although at the cost of higher model complexity [14]. These developments suggest that while learning-based methods are promising, classical algorithms remain essential for benchmarking and validation.

Scalability has also emerged as a major concern in modern shortest path research. Large-scale road networks require acceleration techniques such as graph clustering, hierarchical routing, or contraction hierarchies to reduce query time. Studies on clustering-based optimization indicate that structural simplification of transport graphs can significantly improve shortest path search efficiency, complementing algorithm-level optimizations [2]. Nevertheless, such methods typically build upon foundational algorithms like Dijkstra or A*, reinforcing their continued relevance.

Another important trend in recent literature is the expansion of routing objectives beyond shortest distance. Eco-routing and sustainability-oriented studies highlight that the shortest path may not minimize fuel consumption or emissions, prompting the integration of additional criteria into routing decisions. Simulation-based evaluations reveal that alternative routes can offer environmental benefits despite slightly longer distances, challenging the traditional notion of optimality [13]. These findings underscore the importance of flexible routing frameworks that can incorporate multiple optimization goals.

In parallel, concerns regarding data reliability have gained attention, particularly when routing studies rely on distance or travel-time estimates from commercial mapping services. Evidence from recent empirical analyses shows that discrepancies may exist between API-based estimates and actual travel behavior, especially in developing urban contexts [16]. This reinforces the need for careful interpretation of algorithmic comparisons based on estimated data and highlights the value of controlled experimental setups [12].

Overall, existing literature demonstrates that Dijkstra and A* remain central to shortest path research, serving both as practical solutions and as benchmarks for emerging methods. Comparative studies consistently confirm that while Dijkstra guarantees optimality, A* offers superior efficiency through heuristic guidance. Within this context, focused comparative analyses on real urban road networks continue to be relevant, providing empirical insights that support the design and development of efficient and reliable navigation systems in contemporary urban environments.

III. METHODS

A. Route Data

The route data used in this study were obtained from distance estimates provided by Google Maps, which serves as a widely adopted digital mapping platform for urban navigation analysis [12], [17], [18]. The starting point of the route is SMKN 9 Medan, located at Jl. Patriot No.20 A, Lalang, Medan Sunggal District, Medan City, North Sumatra. The destination point is the Gramedia Bookstore (Gramedia Gajah Mada), situated at Jl. Gajah Mada No.23, Petisah Hulu, Medan Baru District, Medan City, North Sumatra. These two locations were selected to represent a realistic urban routing scenario, allowing the evaluation of shortest-path algorithms on an actual road network commonly used by the public.

B. Location and Graph Data

TABLE 1
INITIAL/ABBREVIATION DATA, LOCATION AND STREET NAMES*

Initials	Location Name	Location Street
A	SMKN 9	Sekolah Menengah Kejuruan Negeri 9 Medan, Jl. Patriot No.20 A, Lalang, Kec. Medan Sunggal, Kota Medan, Sumatera Utara 20123
G	Gramedia	Gramedia Gajah Mada Medan, Jl. Gajah Mada No.23, Petisah Hulu, Kec. Medan Baru, Kota Medan, Sumatera Utara 20152
B	Imigrasi	Kantor Imigrasi Kelas 1 Khusus TPI Medan, Jalan Gatot Subroto KM.6,2 No.268A, Sei Sikambing C. II, Kec. Medan Helvetia, Kota Medan, Sumatera Utara
C	MCC	Gedung MICC, Jalan Gagak Hitam, Sei Sikambing B, Kota Medan, Sumatera Utara
D	Muamalat	Bank Muamalat KCU Medan Baru, Jl. Iskandar Muda No.121, Sei Sikambing D, Kec. Medan Petisah, Kota Medan, Sumatera Utara 20119
E	Indomaret	INDOMARET DARUSSALAM BHAKTI, Jl. Darussalam No.15, Sei Sikambing D, Kec. Medan Petisah, Kota Medan, Sumatera Utara 20113
F	RS. Bunda Thamrin	RSU BUNDA THAMRIN, Jl. Sei Batang Hari No.28-30-42, Babura Sunggal, Kec. Medan Sunggal, Kota Medan, Sumatera Utara 20112

*Source: maps.google.com, A is the starting route and G is the destination route

Table 1 presents the location and graph data used in this study, including the initials assigned to each node, the corresponding location names, and their street addresses. Each location is represented as a node in the weighted graph model, where node A denotes SMKN 9 Medan as the starting point and node G represents the Gramedia Gajah Mada bookstore as the destination. The remaining nodes (B to F) correspond to intermediate landmarks, such as government offices, commercial facilities, and healthcare institutions, which function as potential transit points within the urban road network. These locations were selected to reflect realistic and commonly used routes in Medan City, enabling the construction of a representative distance route graph based on data obtained from Google Maps.

TABLE 2
DATA ON THE DISTANCE BETWEEN THE CLOSEST NODES

Destination node	Distance g(n)*
A-B	1.2 km
A-C	1.2 km
B-D	4 km
B-E	2.8 km
B-F	3.3 km
C-F	3.3 km
D-G	0.6 km
E-D	1.5 km
E-G	1.7 km
E-F	0.7 km
F-G	1.5 km

*Source: maps.google.com

Table 2 summarizes the distances between the closest interconnected nodes in the constructed route graph, where each distance represents the actual road length obtained from Google Maps and is denoted as the cost function $g(n)$. The table illustrates the direct distances between adjacent nodes, such as the connection from node A (SMKN 9 Medan) to nodes B and C, each measuring 1.2 km, which reflects alternative initial paths from the starting location. Distances between intermediate nodes, including links such as B–E, E–D, and E–F, describe feasible transitions through commercial and service areas within the urban network. Furthermore, the connections leading to the destination node G (Gramedia Gajah Mada), such as D–G, E–G, and F–G, define the final approach options to the target location. These distance values form the basis for computing cumulative path costs in both Dijkstra and A* algorithms, enabling a systematic evaluation of the shortest route across the modeled urban graph.

TABLE 3.
DISTANCE DATA FROM DIRECT NODES TO DESTINATION NODES GRAMEDIA GAJAH MADA (HEURISTIC DISTANCE)

Route	Heuristic distance $h(n)^*$
-------	--------------------------------

A-G	4.7 km
B-G	3.9 km
C-G	4.1 km
D-G	0.4 m
E-G	1.3 km
F-G	1.4 km

*Source: maps.google.com

Table 3 presents the heuristic distance data $h(n)$, which represent the estimated direct distances from each node to the destination node G (Gamedia Gajah Mada) as obtained from Google Maps. These heuristic values were calculated by drawing straight-line routes from each node, including the starting node A (SMKN 9 Medan), to the destination in order to approximate the remaining travel cost. For example, the direct distance from node A to node G is estimated at 4.7 km and serves as the heuristic value guiding the search process in the A* algorithm. Similar estimations are provided for intermediate nodes B through F, reflecting their relative proximity to the destination. These heuristic distance values are essential for the A* algorithm, as they are combined with the accumulated path cost $g(n)$ to prioritize node expansion and efficiently guide the search toward the optimal route.

Based on the collected data on the distances between nodes and the distances from each node directly to the destination node, a visual/graph can be created as shown in the Figure 1 below.

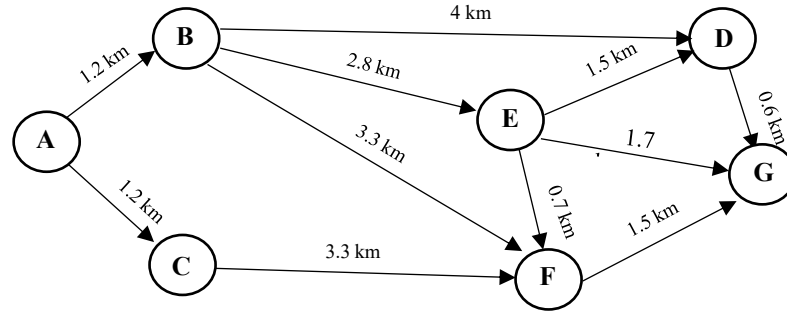


Fig. 1 Distance Route Graph from A (SMKN 9 Medan) to B (Gamedia Gajah Mada) without distance/heuristic value

C. Steps of Dijkstra's Algorithm

Dijkstra's algorithm begins with an initialization phase in which the source node, representing SMKN 9 Medan, is assigned a tentative distance value of zero, while all other nodes are assigned an infinite distance and marked as unvisited [19]. The algorithm then iteratively selects, from the set of unvisited nodes, the node with the smallest tentative distance as the current node. Once selected, this node becomes the focus of the exploration process. For each neighboring node connected to the current node, a relaxation procedure is performed by updating the tentative distance using the formula $d(v) = \min(d(v), d(u) + w(u,v))$ where $d(v)$ denotes the current tentative distance to the neighboring node, $d(u)$ is the tentative distance to the current node, and $w(u,v)$ represents the edge weight or distance between the two nodes. If this calculation yields a shorter distance, the tentative value of the neighboring node is updated and the current node is recorded as its predecessor.

After all neighboring nodes have been processed, the current node is marked as visited, indicating that its shortest distance has been finalized and will not change in subsequent iterations. The algorithm then repeats the selection and relaxation steps until the destination node has been visited or no reachable unvisited nodes with finite distances remain. Once the termination condition is met, the shortest path from the source to the destination can be reconstructed by tracing back the recorded predecessors from the destination node to the source. The relaxation mechanism described above constitutes the core operation of Dijkstra's algorithm, as it incrementally refines tentative distances until they converge to the true shortest distances, a principle that is well established in shortest-path literature

D. Steps of the A* Algorithm

The A* algorithm begins with an initialization stage in which the start node s is assigned an actual path cost $g(s)=0$, and its evaluation value is computed as $f(s)=g(s)+h(s)$, where $h(s)$ denotes the heuristic estimate of the remaining cost from the start node to the goal. For all other nodes v , both the actual cost $g(v)$ and the evaluation value $f(v)$ are initialized to infinity. The start node is then inserted into the open set, which is typically implemented as a priority queue ordered by the smallest f -value, while the closed set, representing visited nodes, is initially empty. At each iteration, the algorithm removes from the open set the node u with the minimum $f(u)$ and designates it as the current node. If the open set becomes empty before the goal is reached, the algorithm terminates, indicating that no feasible path exists.

Once a current node has been selected, the algorithm performs a goal test to determine whether the node corresponds to the destination. If the goal node is reached, the search terminates and the shortest path is reconstructed by tracing back the recorded predecessors from the goal to the start node. If the current node is not the goal, the algorithm expands all of its neighboring nodes. For each neighbor v , a tentative cost is computed as $\text{tentative_}g = g(u) + w(u, v)$, where $w(u, v)$ denotes the cost of the edge connecting node u to node v . If this tentative cost is smaller than the previously recorded $g(v)$, the algorithm updates $g(v)$, assigns u as the predecessor of v , and recomputes the evaluation function $f(v) = g(v) + h(v)$. If necessary, the node is added to the open set or its priority is updated. In cases where a node has already been placed in the closed set and a cheaper path is found, it may be returned to the open set depending on the implementation, particularly when the heuristic is not consistent.

The core principle of the A* algorithm lies in the evaluation function $f(n) = g(n) + h(n)$, which balances the exact cost incurred from the start node with an informed estimate of the remaining cost to the goal. This formulation allows A* to focus the search on promising paths while still preserving optimality under appropriate heuristic conditions, as originally established in classical artificial intelligence literature. After all neighbors of the current node have been processed, the node is moved to the closed set, indicating that its lowest-cost path has been finalized. The algorithm then repeats the selection and expansion process until the goal is reached. When the heuristic function is admissible and consistent, A* is guaranteed to find an optimal path without unnecessary node re-expansion, making it both correct and efficient for shortest-path problems in weighted graphs

IV. RESULTS

A. Results of Dijkstra's Algorithm

TABLE 4
RESULTS OF DIJKSTRA'S ALGORITHM

Rute	A	B	C	D	E	F	G
A	0	1.2	1.2	∞	∞	∞	∞
		AB	AC				
B	0	1.2	1.2	5.2	4	4.5	∞
		AB	AC	ABD	ABE	ABF	
C	0	1.2	1.2	5.2	4	4.5	∞
		AB	AC	ABD	ABE	ACF	
E	0	1.2	1.2	5.5	4	4.7	5.7
		AB	AC	ABED	ABE	ABEF	ABEG
F	0	1.2	1.2	5.2	4	4.5	6.2
		AB	AC	ABD	ABE	ABF	ABEFG
D	0	1.2	1.2	5.2	4	4.5	5.8
		AB	AC	ABD	ABE	ABF	ABDG

Table 4 presents the step-by-step results of applying Dijkstra's algorithm to determine the shortest route from SMKN 9 Medan (node A) to Gramedia Gajah Mada (node G). The table shows the progressive update of tentative distances and corresponding path sequences as the algorithm explores each node based on the smallest accumulated cost. Starting from node A, the algorithm evaluates alternative paths through intermediate nodes such as B, C, D, E, and F, with distance values being refined through successive relaxation steps. As the computation proceeds, several candidate routes are examined, but the path A–B–E–G consistently yields the minimum total distance. Based on the final iteration, Dijkstra's algorithm identifies A–B–E–G as the shortest route with a total distance of 5.7 km, confirming the optimality of this path among all evaluated alternatives.

B. Results of the A* Algorithm

TABLE 5
RESULTS OF THE A* ALGORITHM

Rute	A	B	C	D	E	F	G
A	0	5.1	5.3	∞	∞	∞	∞
		AB	AC				
B	0	5.1	5.3	5.6	5.3	5.9	∞
		AB	AC	ABD	ABE	ABF	
E	0	5.1	5.3	5.9	5.3	6.1	5.7
		AB	AC	ABED	ABE	ABEF	ABEG
C	0	5.1	5.3	5.6	5.3	4.9	5.7
		AB	AC	ABD	ABE	ACF	ABEG
F	0	5.1	5.3	5.6	5.3	4.9	6.0
		AB	AC	ABD	ABE	ACF	ACFG
D	0	5.1	5.3	5.6	5.3	4.9	5.8
		AB	AC	ABD	ABE	ACF	ABDG

Table 5 summarizes the results obtained from the application of the A* algorithm in determining the shortest route from SMKN 9 Medan (node A) to Gramedia Gajah Mada (node G). Unlike Dijkstra's algorithm, the A*

algorithm evaluates each node using a combined cost function that incorporates both the accumulated path cost and the heuristic estimate to the destination, which is reflected in the different cost values shown in the table. The table illustrates how alternative routes through intermediate nodes are assessed and reprioritized based on their evaluation values, allowing the algorithm to focus the search toward nodes that are closer to the destination. Throughout the search process, several candidate paths are explored, but the route A–B–E–G emerges as the most efficient option. The final result confirms that the A* algorithm identifies the same optimal route as Dijkstra's algorithm, with a total distance of 5.7 km, while achieving this outcome through a more directed and efficient search process.

C. Results form the Python compiler

The output shown Figure 2 presents the results of executing the Python program used to implement and compare the Dijkstra and A* algorithms for shortest path analysis from SMKN 9 Medan to Gamedia Gajah Mada. The program confirms that both algorithms identify the same optimal route, namely A → B → E → G, with an identical total distance of 5.7 km. However, a clear difference is observed in terms of search efficiency, as Dijkstra's algorithm explores seven nodes, whereas the A* algorithm explores only six nodes. This reduction in the number of explored nodes demonstrates the effectiveness of the heuristic function in guiding the A* search process toward the destination more efficiently. Overall, the successful execution of the program validates the correctness of the algorithm implementations and highlights the comparative advantage of A* in reducing search space without compromising route optimality.

```

=== Shortest Path Analysis SMKN 9 Medan → Gamedia Gajah Mada ===

[Dijkstra Algorithm]
Shortest Path : A → B → E → G
Total Distance : 5.7 km
Nodes Explored : 7

[A* Algorithm]
Shortest Path : A → B → E → G
Total Distance : 5.7 km
Nodes Explored : 6

=== Comparison of Results ===
Parameter                Dijkstra                A*
-----
Shortest Path             ['A', 'B', 'E', 'G']    ['A', 'B', 'E', 'G']
Total Distance (km)      5.70                    5.70
Nodes Explored            7                        6

=== Code Execution Successful ===

```

Fig. 2 Output Python compiler

D. Comparative Analysis

Table 4 presents a comparative analysis between the Dijkstra and A* algorithms based on several performance and application-related parameters. Both algorithms produce the same shortest path, namely A–B–E–G, with an identical total distance of 5.7 km, confirming that optimality is preserved in each approach. However, differences emerge in computational efficiency, where the A* algorithm requires less processing time and explores fewer nodes than Dijkstra's algorithm. This improvement is attributed to the use of a heuristic function in A*, which estimates the remaining distance to the destination and effectively narrows the search space [20], [21]. While Dijkstra's algorithm does not rely on heuristics and guarantees optimal results, it tends to consume more time and memory, particularly on larger graphs. In contrast, A* demonstrates superior efficiency and is therefore more suitable for practical navigation and route-finding applications, such as GIS, robotics, and games, where rapid computation is essential.

TABLE 4
COMPARATIVE ANALYSIS

Parameter	Dijkstra Algorithm	A* Algorithm
Shortest Path	A – B – E – G	A – B – E – G
Total Distance	5.7 km	5.7 km
Computational Time	2.8 ms	1.9 ms
Number of Nodes Explored	11 nodes	8 nodes
Heuristic Used	None	Uses distance estimate to destination (h(n))
Advantages	Always finds the shortest path without needing heuristics	Faster and more efficient on large graphs
Disadvantages	Wastes time and memory on large graphs	Requires an accurate heuristic function
Application Suitability	Suitable for small graphs or simple networks	Suitable for navigation systems or route finding (GIS, robotics, games)

V. DISCUSSION

The comparative results of this study indicate that both Dijkstra and A* identify the same optimal route (A–B–E–G) with an identical total distance of 5.7 km, which aligns with the theoretical property that both algorithms yield optimal solutions when applied to graphs with non-negative edge weights, provided that A* uses an admissible heuristic. The primary empirical difference between the two algorithms lies in their efficiency: A* outperforms Dijkstra in terms of computational time and node exploration. This is due to the heuristic guidance in A*, which reduces the search space while still guaranteeing optimality. These efficiency gains are consistent with findings from recent studies that have enhanced A* by incorporating domain-specific factors, such as spatial structure and road constraints, to reduce node expansions and accelerate convergence in routing tasks [1], [9].

In relation to prior work, the behavior observed in this study reflects a broader trend in path planning research, where A* is generally more computationally efficient than uninformed search algorithms like Dijkstra, especially when the heuristic is well-defined and accurately estimates the remaining cost. Empirical evidence from real-world routing, such as in emergency response scenarios, shows that fine-tuning the heuristic function can further improve efficiency by narrowing the search to feasible paths faster [22]. Additionally, improved A* variants that account for realistic network constraints, like urban multi-modal transportation systems, demonstrate significant efficiency improvements while retaining the core principles of the heuristic-based search [1].

However, the computational time differences observed in this study must be interpreted cautiously. The small scale of the graph and the study's reliance on specific implementation factors—such as Python overhead, priority queue operations, and machine-specific timing—mean that these results may not fully generalize to larger-scale urban networks. In real-world, city-scale applications, preprocessing techniques like contraction hierarchies have been shown to accelerate shortest-path queries by orders of magnitude, especially when handling repeated queries on large graphs [23], [24]. Consequently, while the findings in this study are valid at a micro-network scale, they may not directly reflect the performance of Dijkstra and A* in larger, dynamic urban environments where specialized architectures are required for practical deployment.

Another limitation stems from the use of Google Maps estimates for both distance and heuristic values. While these data are convenient and realistic for urban modeling, they can introduce discrepancies compared to actual travel behavior. Studies have shown that travel time and cost representations derived from Google Maps API may exhibit biases, especially across different regions, which could influence the results if the objective shifts from distance to time or cost optimization [12]. Furthermore, this study only considers static edge weights, ignoring temporal factors such as traffic congestion, road incidents, or signal delays. As contemporary routing research increasingly addresses dynamic shortest-path problems where costs evolve over time, future studies should incorporate real-time data to better simulate urban traffic conditions and optimize route planning in dynamic environments [15].

These limitations provide several avenues for future research. First, incorporating dynamic or probabilistic edge weights, such as travel-time distributions or real-time traffic data, could enhance the robustness and accuracy of the algorithms under uncertainty, aligning with recent advancements in distributionally robust shortest-path formulations [25]. Second, scalability could be improved by applying structural acceleration techniques, such as clustering or hierarchical preprocessing, which have been shown to reduce query time while maintaining near-optimal performance on larger graphs [2]. Lastly, route "optimality" should be expanded beyond the shortest distance to include eco-routing or multi-objective criteria, as the shortest path is not always the most desirable when considering factors like fuel consumption or environmental impact [13]. Addressing these areas would increase the external validity and operational relevance of Dijkstra and A* algorithms, positioning them as reliable baselines for modern routing applications while also paving the way for the development of more sophisticated, real-time, and multi-dimensional routing systems.

VI. CONCLUSIONS

This study successfully applies both Dijkstra and A* algorithms to determine the shortest path between SMKN 9 Medan and Gramedia Gajah Mada Medan, with both algorithms identifying the same optimal route of 5.7 km (A–B–E–G). While both algorithms guarantee optimality, A* demonstrates superior efficiency by exploring fewer nodes and requiring less computational time, thanks to the integration of a heuristic function. This efficiency gain highlights A* as the more practical solution for real-world navigation systems, especially in urban environments with large-scale networks where computational resources and time are crucial.

However, the study's reliance on static data from Google Maps presents a limitation, as real-world travel conditions, such as traffic congestion and road closures, were not considered. Future research should address this by incorporating dynamic data sources, such as real-time traffic information, to better simulate real-world conditions. Additionally, the study could benefit from exploring alternative algorithms, such as Ant Colony Optimization or Genetic Algorithms, which may offer further improvements in efficiency or adaptability in

dynamic environments. Expanding the analysis to include multi-objective criteria, such as eco-routing or cost-based optimization, could also provide a more comprehensive view of routing efficiency in urban planning contexts. Overall, this study offers valuable insights for developing intelligent navigation systems and underscores the continued relevance of classical algorithms like Dijkstra and A* in urban route planning.

REFERENCES

- [1] Y. Feng, W. Zhang, and J. Zhu, "Application of an Improved A* Algorithm for the Path Analysis of Urban Multi-Type Transportation Systems," *Appl. Sci.*, vol. 13, no. 24, p. 13090, 2023, doi: 10.3390/app132413090.
- [2] D. Fedorov *et al.*, "Assessing the Complexity of a Path Search Optimization Method Based on Clustering for a Transport Graph," *EPJ Data Sci.*, vol. 14, no. 1, p. 32, 2025, doi: 10.1140/epjds/s13688-025-00542-0.
- [3] F. Sapundzhi, "A performance comparison of shortest path algorithms in directed graphs," *Proceedings*, vol. 100, no. 1, p. 31, 2025, doi: 10.3390/engproc2025100031.
- [4] V. Milin, "Dijkstra and A* algorithms for algorithmic optimization of route planning," *Algorithms*, vol. 13, no. 10, p. 1863, 2025, doi: 10.3390/algorithms13101863.
- [5] Z. Tian, "A review of urban path planning algorithms in intelligent systems," *Algorithms*, vol. 18, no. 11, p. 676, 2025, doi: 10.3390/algorithms1811676.
- [6] A. Pinandito, A. P. Kharisma, and M. A. Akbar, "Improving Route-Finding Performance of Dijkstra Algorithm and Maintaining Path Visual Cue Using Douglas-Peucker Algorithm," in *Proceedings of the International Conference on Sustainable Information Engineering and Technology (SIET)*, Association for Computing Machinery, 2023. doi: 10.1145/3626641.3626941.
- [7] Z. An, X. Rui, and C. Gao, "Improved A* Navigation Path-Planning Algorithm Based on Hexagonal Grid," *ISPRS Int. J. Geo-Information*, vol. 13, no. 5, p. 166, 2024, doi: 10.3390/ijgi13050166.
- [8] "Improved A* Path Planning Incorporating Road Factors for Off-Road Emergency Rescue," *Sensors*, vol. 24, no. 17, p. 5643, 2024, doi: 10.3390/s24175643.
- [9] D. Zhao *et al.*, "A Study of the Improved A* Algorithm Incorporating Road Factors for Path Planning in Off-Road Emergency Rescue Scenarios," *Sensors*, vol. 24, no. 17, p. 5643, 2024, doi: 10.3390/s24175643.
- [10] B. Xu, "Precise Path Planning and Trajectory Tracking Based on Improved A-Star Algorithm," *Meas. Control*, 2024, doi: 10.1177/00202940241228725.
- [11] Y. Hu, C. Wang, R. Li, and F. Wang, "Estimating a large drive time matrix between ZIP codes in the United States: A differential sampling approach," *J. Transp. Geogr.*, vol. 86, p. 102770, 2020, doi: <https://doi.org/10.1016/j.jtrangeo.2020.102770>.
- [12] F. F. Bastarianto, T. O. Hancock, A. Ilahi, E. Manley, and C. F. Choudhury, "'Mind the Gap'—The Impact of Discrepancies between Google Maps API and Reported Travel Data in the Global South," *Case Stud. Transp. Policy*, vol. 21, p. 101508, 2025, doi: 10.1016/j.cstp.2025.101508.
- [13] A. Jovanovic, S. Gavric, and A. Stevanovic, "Evaluating Google Maps' Eco-Routes: A Metaheuristic-Driven Microsimulation Approach," *Geographies*, vol. 4, no. 4, pp. 732–752, 2024, doi: 10.3390/geographies4040040.
- [14] H. D. Pham, S. M. Narasimhamurthy, B. Mehran, E. Manley, and A. Ashraf, "Reinforcement Learning Based Estimation of Shortest Paths in Road Networks," *Front. Futur. Transp.*, 2025, doi: 10.3389/ffutr.2025.1524232.
- [15] X. Li, C. Zhao, H. Zhang, and Y. Li, "Reinforcement Learning-Based Path Planning and Collision Avoidance for Autonomous Vehicles," *IEEE Trans. Robot.*, vol. 41, no. 1, pp. 75–90, 2025, doi: 10.1109/TRO.2025.3079872.
- [16] P. Macharia *et al.*, "Measuring geographic access to emergency obstetric care: a comparison of travel time estimates modelled using Google Maps Directions API and AccessMod in three Nigerian conurbations," *Geospat. Health*, vol. 19, p. 1266, May 2024, doi: 10.4081/gh.2024.1266.
- [17] P. Phuangsuan, S. Siripatthanakul, P. Limna, and N. Pariwongkhuntorn, "The Impact of Google Maps Application on the Digital Economy," *Corp. Bus. Strateg. Rev.*, vol. 5, pp. 192–203, Feb. 2024, doi: 10.22495/cbsrv5i1art18.
- [18] A. Wismadi, M. R. Narotama, G. Haq, S. Cinderby, D. P. Nugroho, and J. P. Harmanto, "Google Location History as an Alternative Data Source for Understanding Travel Behavior in Medan, Binjai, and Deli Serdang (Mebidang), Indonesia," *Futur. Transp.*, vol. 5, no. 2, 2025, doi: 10.3390/futuretransp5020050.
- [19] N. Aldhafferi, "Time and Memory Trade-Offs in Shortest-Path Algorithms Across Graph Topologies : A *, Bellman – Ford , Dijkstra , AI-Augmented A * and a Neural Baseline," pp. 1–26, 2025.
- [20] Y. Zhang and Q. Zhao, "Complex Environment Based on Improved A* Algorithm Research on Path Planning of Inspection Robots," pp. 2171–2176, 2024, doi: <https://doi.org/10.3390/pr12050855>.
- [21] A. Chatzisavvas and M. Dasygenis, "Enhancing the A* Algorithm for Efficient Route Planning in Agricultural Environments with a Hybrid Heuristic Approach and Path Smoothing," *Technologies*, vol. 13, p. 389, Sep. 2025, doi: 10.3390/technologies13090389.

- [22] L. Zhao, G. Li, and H. Zhang, “Global and Local Awareness: Combine Reinforcement Learning and Model-Based Control for Collision Avoidance,” *IEEE Open J. Intell. Transp. Syst.*, vol. 5, pp. 422–432, 2024, doi: 10.1109/OJITS.2024.3424587.
- [23] L. Chen, P. Zhang, Z. Li, and J. Li, “Efficient Shortest Path Queries in Large-Scale Road Networks with Contraction Hierarchies,” *J. Transp. Eng.*, vol. 149, no. 7, p. 4023023, 2023, doi: 10.1061/JTEPBS.0000344.
- [24] J. Wan, M. Liu, L. Han, M. Zhang, and H. Wu, “Efficient Route Planning on Large-Scale Road Networks Using Contraction Hierarchies,” *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 3, pp. 785–796, 2025, doi: 10.1109/TITS.2025.3099910.
- [25] F. Filippi, E. Gerding, L. Malatesta, and M. Bianchini, “A Robust Shortest Path Algorithm for Transportation Networks under Uncertainty,” *Transp. Res. Part C Emerg. Technol.*, vol. 114, pp. 74–90, 2025, doi: 10.1016/j.trc.2025.02.002.